

Module 2:Transport Layer and Network Layer



Module 2: Transport Layer and Network Layer

Transport Layer

- Services and Protocols
- UDP and TCP

Hands-on

- Sockets Introduction
- Elementary TCP Sockets
- TCP Client/Server Example
- I/O Multiplexing
 - select()
 - poll()
- Elementary UDP Sockets
- Advanced I/O Functions

Network Layer

- Introduction
- Network-layer Protocols
- Unicast Routing
- Multicast Routing
- Multicasting Basics
- Intra-domain Routing
- Inter-domain Routing
- Next Generation IP (IPv6)
- Quality of Service (QoS)

Hands-on

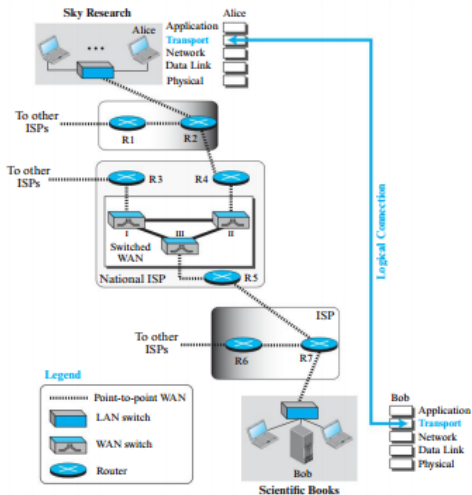
- Linux Routing Table
- Routing Caches
- Routing Cache Implementation
- Adding Routes using ip

Introduction to the Transport Layer

- The **Transport Layer** is located between:
 - Application Layer
 - Network Layer
- It provides **process-to-process communication** between:
 - A local application process
 - A remote application process
- Communication is achieved using a **logical connection**
- A logical connection means:
 - Applications assume a direct connection exists
 - Messages can be sent and received as if directly connected

Logical connection at the transport layer

Figure 3.1 Logical connection at the transport layer



Logical Connection in the Transport Layer

Alice's Host $\longleftrightarrow$ Bob's Host

- Example:
 - Alice at Sky Research Company
 - Bob at Scientific Books Company
- Both hosts communicate as though a direct connection exists between them
- In reality:
 - Data passes through multiple routers and networks
 - Routers use only the first three layers:
 - Physical Layer
 - Data Link Layer
 - Network Layer
- Only the **end systems** use the Transport Layer services

Transport-Layer Services

- The **Transport Layer** is located between:
 - Application Layer
 - Network Layer
- Responsibilities:
 - Provides services to the Application Layer
 - Receives services from the Network Layer
- Major Services Provided by the Transport Layer:
 - 1 Process-to-Process Communication
 - 2 Addressing using Port Numbers
 - 3 Encapsulation and Decapsulation
 - 4 Multiplexing and Demultiplexing
 - 5 Flow Control
 - 6 Error Control
 - 7 Congestion Control
 - 8 Connectionless Services
 - 9 Connection-Oriented Services

Process-to-Process Communication

- The first duty of the **Transport Layer** is to provide:
 - **Process-to-Process Communication**
- A **process** is:
 - An application-layer entity
 - A running program using transport-layer services
- Examples of processes:
 - Web browser
 - Email application
 - File transfer application

Host-to-Host vs Process-to-Process Communication(Process-to-Process Communication)

Network Layer

- Provides:
 - Host-to-Host Communication
- Delivers data:
 - Only to the destination computer
- Communication at:
 - Computer level

Transport Layer

- Provides:
 - Process-to-Process Communication
- Delivers data:
 - To the correct process
- Communication at:
 - Application level

Important Point

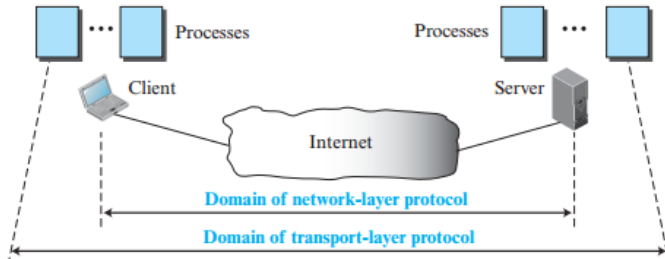
Delivery to the destination host alone is incomplete.
The message must also reach the correct application process.

Role of the Transport Layer(Process-to-Process Communication)

- The Network Layer identifies:
 - The destination computer (host)
- The Transport Layer identifies:
 - The destination process or application
- Therefore:
 - Network Layer → Host Delivery
 - Transport Layer → Process Delivery
- Example:
 - A computer may run:
 - Web server
 - Email server
 - FTP server
 - Transport Layer ensures data reaches the correct application

Network layer versus transport layer(Process-to-Process Communication)

Figure 3.2 *Network layer versus transport layer*

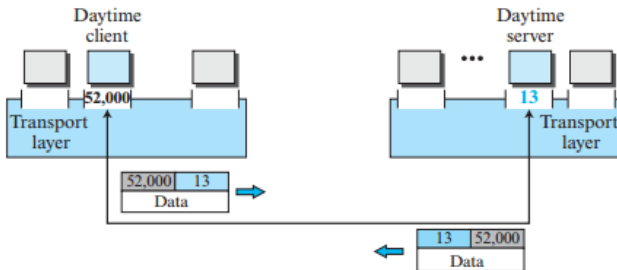


Addressing: Port Numbers — Client-Server Communication

- Process-to-process communication commonly uses the:
 - **Client-Server Paradigm**
- **Client**
 - Requests services
 - Usually runs on the local host
- **Server**
 - Provides services
 - Usually runs on the remote host
- Example:
 - Daytime Client requests date and time
 - Daytime Server sends date and time

Port Numbers

Figure 3.3 *Port numbers*



Addressing: Port Numbers — Need for Port Numbers

- Modern operating systems support:
 - Multiuser environments
 - Multiprogramming environments
- A computer can run:
 - Multiple server programs
 - Multiple client programs
- Communication requires identification of:
 - Local host
 - Remote host
 - Local process
 - Remote process
- Hosts are identified using:
 - IP Addresses
- Processes are identified using:
 - **Port Numbers**

Addressing: Port Numbers — Port Number Range

- In TCP/IP, port numbers are:
 - 16-bit integers
 - Range: 0 to 65,535
- Port numbers uniquely identify:
 - Application processes on a host

Important Idea

IP Address identifies the host

Port Number identifies the process

Addressing: Port Numbers — Ephemeral Port Numbers

- Client programs use:
 - **Ephemeral Port Numbers**
- Meaning of Ephemeral:
 - Short-lived or temporary
- Characteristics:
 - Assigned temporarily
 - Usually greater than 1023
 - Used during client communication
- Example:
 - Client Port Number = 52000

Addressing: Port Numbers — Well-Known Port Numbers

- Server processes use:
 - **Well-Known Port Numbers**
- Why not random numbers?
 - Clients must know the server port number
 - Random numbers create extra overhead
- Well-known ports are:
 - Permanent
 - Universally defined
- Example:
 - Daytime Server uses Port 13

Addressing: Port Numbers — IP Address vs Port Number

IP Address	Port Number
Identifies Host	Identifies Process
Host Selection	Process Selection
Used by Network Layer	Used by Transport Layer

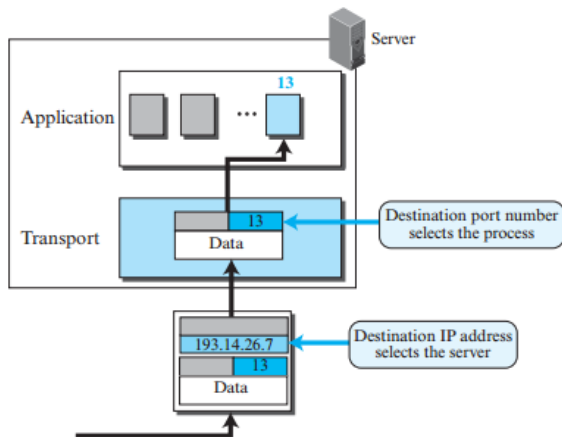
Example

192.168.1.10 : 80

- 192.168.1.10 → Destination Host
- Port 80 → Web Server Process

IP addresses versus port numbers

Figure 3.4 *IP addresses versus port numbers*



Addressing: Port Numbers — ICANN Port Ranges

- ICANN:
 - Internet Corporation for Assigned Names and Numbers
- ICANN divides port numbers into three categories:
 - 1 Well-Known Ports
 - 2 Registered Ports
 - 3 Dynamic (Private) Ports



Port Range	Category
0 – 1023	Well-Known Ports
1024 – 49151	Registered Ports
49152 – 65535	Dynamic/Private Ports

Addressing: Port Numbers — Well-Known Ports

- Range:
 - 0 – 1023
- Characteristics:
 - Assigned and controlled by ICANN
 - Used by standard server applications
- Examples:
 - HTTP → Port 80
 - HTTPS → Port 443
 - FTP → Port 21
 - DNS → Port 53

Addressing: Port Numbers — Registered Ports

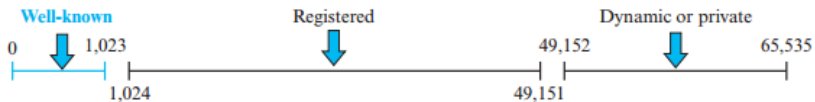
- Range:
 - 1024 – 49151
- Characteristics:
 - Not assigned directly by ICANN
 - Can be registered to avoid duplication
- Commonly used by:
 - User applications
 - Vendor-specific services

Addressing: Port Numbers — Dynamic/Private Ports

- Range:
 - 49152 – 65535
- Characteristics:
 - Neither controlled nor registered
 - Used temporarily by clients
- Also called:
 - Ephemeral Ports
 - Private Ports
- Mainly used for:
 - Temporary client communication

ICANN ranges

Figure 3.5 *ICANN ranges*



Addressing: Port Numbers — Example 3.1

- In UNIX systems, well-known ports are stored in:

File

/etc/services

- Each line in the file contains:
 - Server/Application name
 - Well-known port number
 - Protocol type (TCP/UDP)
- The `grep` utility is used to search entries

Addressing: Port Numbers — TFTP Port Example

Command

```
$ grep tftp /etc/services
```

Output

```
tftp    69/tcp  
tftp    69/udp
```

- TFTP:
 - Trivial File Transfer Protocol
- Uses:
 - Port 69
- Can operate using:
 - TCP
 - UDP

Addressing: Port Numbers — SNMP Port Example

Command

```
$ grep snmp /etc/services
```

Output

```
snmp      161/tcp  # Simple Net Mgmt Proto
snmp      161/udp  # Simple Net Mgmt Proto
snmptrap  162/udp  # Traps for SNMP
```

- SNMP:
 - Simple Network Management Protocol
- Uses two port numbers:
 - Port 161 → Normal SNMP communication
 - Port 162 → SNMP trap messages

Addressing: Port Numbers — Socket Addresses

- A transport-layer connection requires:
 - IP Address
 - Port Number
- The combination of:

Socket Address

IP Address + Port Number

- A socket address uniquely identifies:
 - A process on a host

Addressing: Port Numbers — Client and Server Socket Addresses

- Two socket addresses are needed for communication:
 - 1 Client Socket Address
 - 2 Server Socket Address
- Client Socket Address:
 - Identifies the client process uniquely
- Server Socket Address:
 - Identifies the server process uniquely

Client $\longleftrightarrow$ **Server**
(IP + Port) (IP + Port)

Addressing: Port Numbers — Components of Communication

- Internet communication requires four pieces of information:

- ① Source IP Address
- ② Destination IP Address
- ③ Source Port Number
- ④ Destination Port Number

- These values are stored in:

- Network-layer header → IP Addresses
- Transport-layer header → Port Numbers

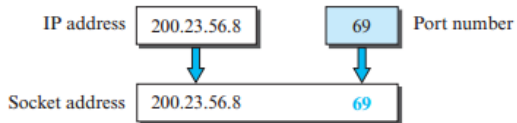
Example

192.168.1.10 : 52000 → Client Socket Address

172.16.1.5 : 80 → Server Socket Address

Socket Address

Figure 3.6 *Socket address*



Encapsulation and Decapsulation — Introduction

- To send data between processes, the Transport Layer performs:
 - ① Encapsulation
 - ② Decapsulation
- Encapsulation:
 - Happens at the sender side
- Decapsulation:
 - Happens at the receiver side

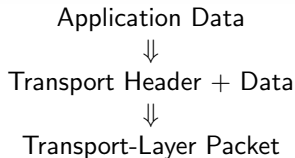


Encapsulation and Decapsulation — Encapsulation Process

- At the sender side:
 - ① Application process creates a message
 - ② Message is passed to the Transport Layer
 - ③ Socket addresses and protocol information are added
 - ④ Transport Layer adds a Transport Header
- This process is called:

Encapsulation

Adding protocol headers to the data before transmission



Encapsulation and Decapsulation — Transport-Layer Packets

- Transport-layer packets have different names depending on the protocol:

Protocol	Packet Name
UDP	User Datagram
TCP	Segment
General Term	Packet

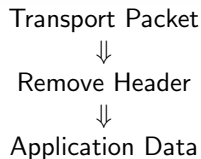
- These packets are forwarded to the Network Layer for transmission

Encapsulation and Decapsulation — Decapsulation Process

- At the receiver side:
 - ① Packet reaches the Transport Layer
 - ② Transport Header is removed
 - ③ Original message is extracted
 - ④ Message is delivered to the correct process
- This process is called:

Decapsulation

Removing protocol headers to recover the original data

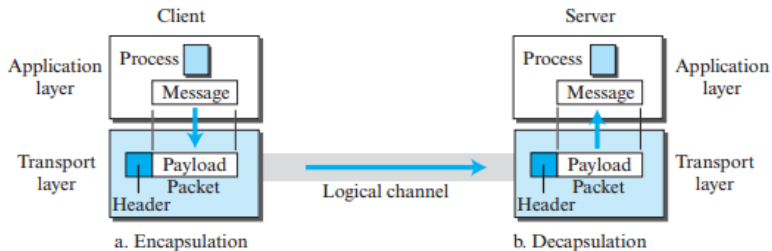


Encapsulation and Decapsulation — Important Points

- The sender socket address is also delivered to the receiver process
- This allows the receiver:
 - To identify the sender
 - To send a reply if needed
- Encapsulation and decapsulation ensure:
 - Proper communication between applications
 - Reliable transfer of messages
 - Correct process delivery

Encapsulation and decapsulation

Figure 3.7 *Encapsulation and decapsulation*



Multiplexing and Demultiplexing — Introduction

- **Multiplexing**

- Many-to-One process
- An entity accepts data from multiple sources

- **Demultiplexing**

- One-to-Many process
- An entity delivers data to multiple destinations

- In the Transport Layer:

- Source side performs Multiplexing
- Destination side performs Demultiplexing

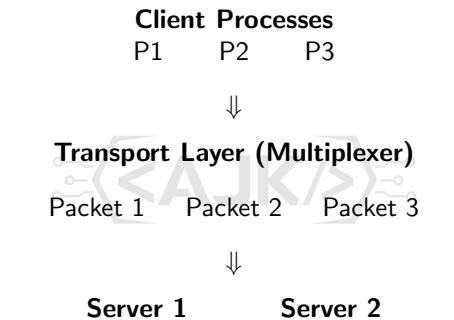
Multiplexing and Demultiplexing — Multiplexing at Source

- Multiple client processes can send data simultaneously
- Example:
 - Client processes:
 - P1
 - P2
 - P3
- The Transport Layer:
 - Accepts messages from all processes
 - Creates separate packets
 - Sends packets through the network
- Therefore, the Transport Layer acts as:

Multiplexer

Combines messages from many processes into packets

Multiplexing and Demultiplexing — Communication Example



- Packets 1 and 3 go to Server 1
- Packet 2 goes to Server 2

Multiplexing and Demultiplexing — Demultiplexing at Destination

- At the destination:
 - Transport Layer receives packets
 - Packets are delivered to correct processes
- The Transport Layer acts as:

Demultiplexer

Separates incoming packets and delivers them to appropriate processes

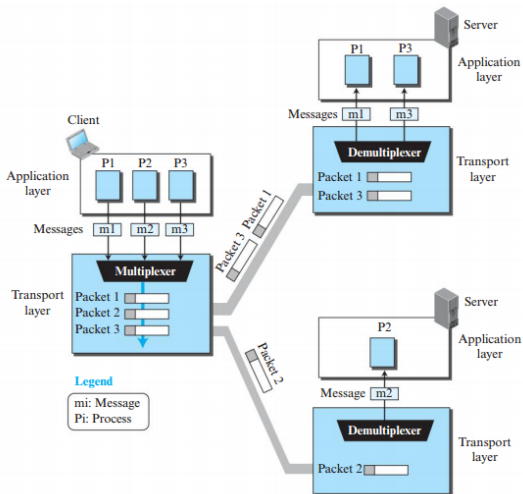
- Example:
 - Packet 1 → Process P1
 - Packet 3 → Process P3

Multiplexing and Demultiplexing — Important Points

- Multiplexing:
 - Many processes share the same communication channel
- Demultiplexing:
 - Incoming packets are delivered to correct applications
- Even if only one message arrives:
 - Demultiplexing is still required
- Port numbers help:
 - Identify the correct destination process

Multiplexing and Demultiplexing

Figure 3.8 Multiplexing and demultiplexing



Flow Control

Definition

Flow Control is the mechanism that maintains a balance between the rate at which the sender produces data and the rate at which the receiver consumes it.

Need for Flow Control

- Prevents receiver buffer overflow.
- Prevents packet loss due to excessive transmission.
- Ensures efficient communication.
- Matches producer and consumer speeds.

Key Point

If data are produced faster than they are consumed, the receiver may become overwhelmed and discard packets.

Pushing and Pulling

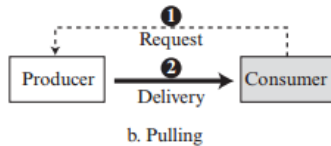
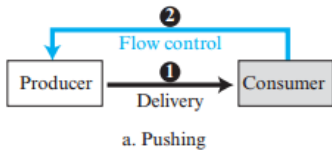
Push Model

- Producer sends data immediately.
- No prior request.
- Receiver may become overloaded.
- Flow control is required.

Pull Model

- Consumer requests data.
- Producer sends only when requested.
- Receiver controls transmission.
- No flow control needed.

Figure 3.9 *Pushing or pulling*



Flow Control at the Transport Layer

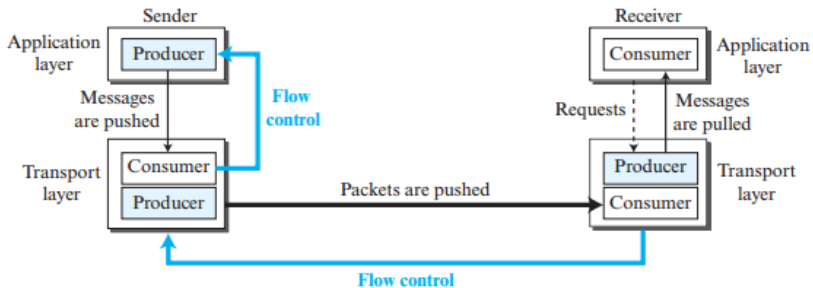
Four entities participate in transport-layer communication.

- ① Sending Application Process
- ② Sending Transport Layer
- ③ Receiving Transport Layer
- ④ Receiving Application Process

Roles

- Application layer acts as a producer.
- Sending transport layer acts as both consumer and producer.
- Receiving transport layer also acts as consumer and producer.
- Receiving application layer normally pulls data.

Figure 3.10 *Flow control at the transport layer*



Buffers for Flow Control

Buffers

Flow control is commonly implemented using

- Sender Buffer
- Receiver Buffer

Sender Buffer

- Stores outgoing packets.
- Stops accepting messages when full.
- Allows transmission when space becomes available.

Receiver Buffer

- Stores incoming packets.
- Stops sender when full.
- Allows sender to continue after vacancies occur.

Example 3.2 and Error Control

Example 3.2

- Sender and receiver each use a single-slot buffer.
- Empty sender slot requests next message.
- Empty receiver slot sends ACK requesting next packet.
- Simple but inefficient.

Error Control

Since the Internet network layer (IP) is unreliable, the transport layer provides reliability whenever required by the application.

Next Topic

Error Control provides reliable delivery by detecting, recovering, and correcting transmission errors.

Definition

Error Control provides reliable communication by detecting, recovering, and correcting transmission errors.

Responsibilities of Error Control

- 1 Detect corrupted packets.
- 2 Discard corrupted packets.
- 3 Detect lost packets and retransmit them.
- 4 Detect duplicate packets.
- 5 Buffer out-of-order packets until missing packets arrive.

Key Point

Unlike Flow Control, Error Control operates only between the sending and receiving transport layers.

Error Control at the Transport Layer

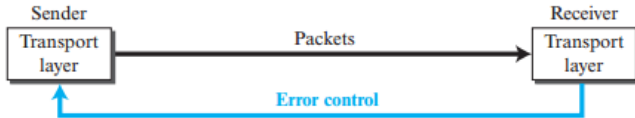
Sender Transport Layer

- Sends numbered packets.
- Stores copies of transmitted packets.
- Retransmits lost packets.

Receiver Transport Layer

- Detects errors.
- Detects duplicates.
- Detects missing packets.
- Sends acknowledgments.

Figure 3.11 *Error control at the transport layer*



Sequence Numbers

Purpose

Each transport-layer packet is assigned a **Sequence Number**.

Sequence Numbers Help To

- Identify lost packets.
- Detect duplicate packets.
- Detect out-of-order packets.
- Request retransmission of missing packets.

If the sequence number field has

m

bits,
the sequence numbers range from

0 to $2^m - 1$

Modulo- 2^m Sequence Numbers

Sequence numbers wrap around after the maximum value.

Example

If

$$m = 4$$

then


$$0, 1, 2, \dots, 15, 0, 1, 2, \dots$$

This numbering system is called

Modulo 2^m

Exam Point

For Error Control, sequence numbers always follow Modulo- 2^m arithmetic.

Acknowledgments

Acknowledgment (ACK)

The receiver informs the sender that packets have been received correctly by sending an **Positive Acknowledgment (ACK)**.

Functions of ACK

- Confirms successful packet reception.
- Prevents unnecessary retransmissions.
- Helps sender determine the next packet.
- Supports reliable communication.

Key Point

Transport-layer protocols generally use **positive acknowledgments** instead of negative acknowledgments.

Combination of Flow Control and Error Control

Concept

Flow Control and Error Control can be combined by using **two numbered buffers**, one at the sender and one at the receiver.

Sender Buffer

- Stores outgoing packets.
- Buffer index becomes the sequence number.
- Copy of each transmitted packet is retained.
- Packet is removed after ACK is received.

Receiver Buffer

- Stores received packets.
- Packet is placed according to its sequence number.
- Sends ACK after successful reception.
- Delivers packets to the application layer.

Key Point

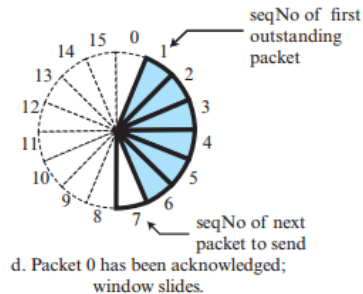
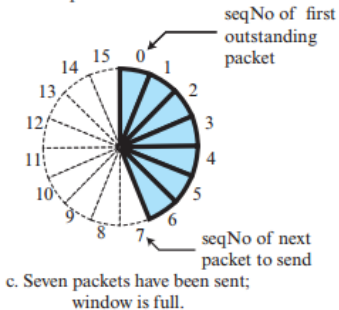
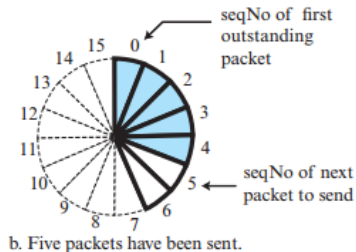
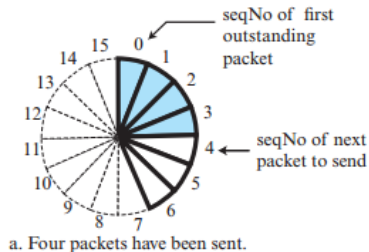
Numbered buffers simultaneously support **Flow Control** and **Error Control**.

Sliding Window

A **Sliding Window** is a buffer-based mechanism that combines flow control and error control by managing packet transmission using sequence numbers.

- Sequence numbers use modulo- 2^m arithmetic.
- Window occupies a portion of the sequence number space.
- Sent packets are marked.
- ACKs unmark corresponding packets.
- When consecutive packets are acknowledged, the window slides forward, allowing new packets to be transmitted.

Figure 3.12 *Sliding window in circular format*



Linear Representation of Sliding Window

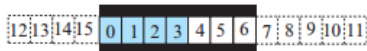
Features

- Easier to represent on paper.
- Equivalent to the circular representation.
- Uses sender variables to track:
 - Next packet to send.
 - Last acknowledged packet.
- Window continuously slides as ACKs arrive.

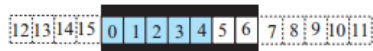
Exam Point

Circular and linear representations describe the same Sliding Window mechanism.

Figure 3.13 *Sliding window in linear format*



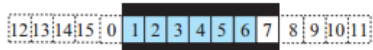
a. Four packets have been sent.



b. Five packets have been sent.



c. Seven packets have been sent;
window is full.



d. Packet 0 has been acknowledged;
window slides.

Congestion Control

Definition

Congestion Control refers to the mechanisms that keep the network load below its capacity to prevent performance degradation.

Causes of Congestion

- Packet arrival rate exceeds network capacity.
- Router and switch buffers become full.
- Input and output queues overflow.
- Increased delay and packet loss.

Observation

- Congestion originates at the Network Layer.
- Its effects are observed at the Transport Layer.
- TCP implements congestion control when the network layer does not provide it.

Key Takeaways

- Numbered buffers combine Flow and Error Control

Connectionless Service

Definition

In a **connectionless transport service**, each message chunk is treated as an independent packet with no relationship to other packets.

Working

- Application divides the message into chunks.
- Chunks are delivered independently.
- Each packet is encapsulated and transmitted separately.
- No logical connection exists between sender and receiver.

Key Point

Each packet is transmitted independently without coordinating with previous or future packets.

Characteristics of Connectionless Service

Problems

- Packets may arrive out of order.
- Packets may experience different delays.
- Lost packets cannot be detected.
- Receiver delivers packets immediately.
- Original message order may be destroyed.

Example

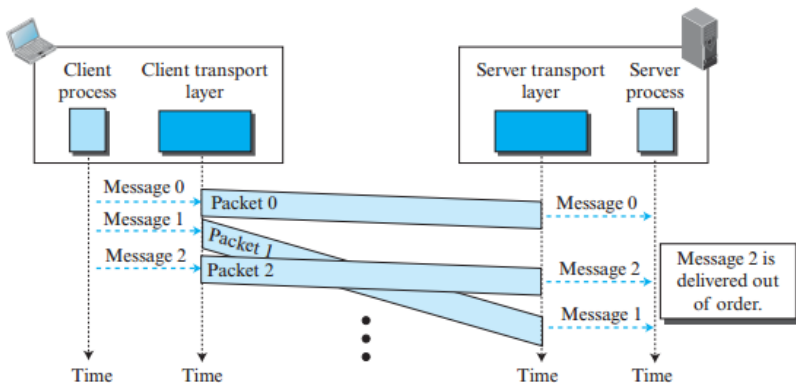
Sender transmits packets:

$0 \rightarrow 1 \rightarrow 2$

Receiver may receive:

$0 \rightarrow 2 \rightarrow 1$

Figure 3.14 *Connectionless service*



Limitations of Connectionless Service

Since packets are independent,

- No packet numbering relationship.
- No coordination between transport layers.
- Receiver cannot identify missing packets.
- Lost packets remain undetected.
- Out-of-order delivery is possible.

Important

Connectionless transport cannot effectively implement

- Flow Control
- Error Control
- Congestion Control

Connection-Oriented Service

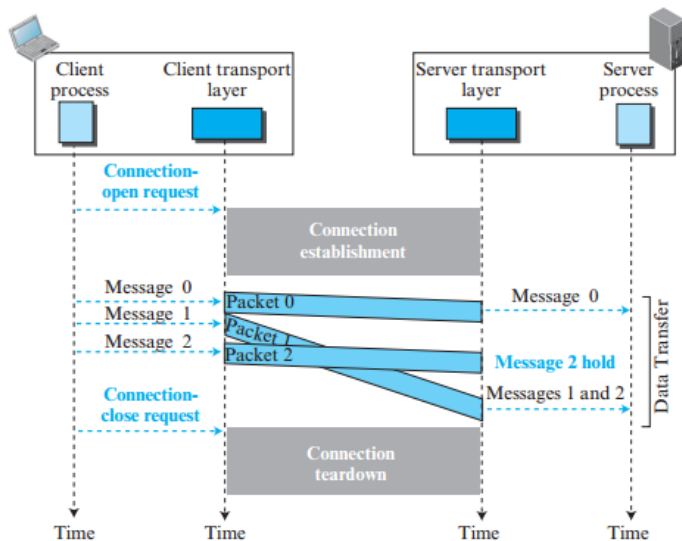
Definition

A **connection-oriented transport service** establishes a logical connection before transmitting data.

Communication Phases

- 1 Connection Establishment
- 2 Data Transfer
- 3 Connection Termination (Tear-down)

Figure 3.15 *Connection-oriented service*



Connection-Oriented vs Connectionless Service

Connectionless

- No setup
- Independent packets
- No reliability
- No ordering guarantee
- No Flow Control
- No Error Control
- Faster

Connection-Oriented

- Connection setup required
- Ordered delivery
- Reliable communication
- Flow Control
- Error Control
- Congestion Control
- Connection teardown

Key Point

Connection-oriented transport protocols provide reliable, ordered, end-to-end communication between hosts.

Finite State Machine (FSM)

Definition

A **Finite State Machine (FSM)** is a model used to describe the behavior of a transport-layer protocol.

It represents the protocol as a machine with a finite number of states, where transitions occur when events take place.

Purpose

- Describes protocol behavior.
- Defines actions for each event.
- Determines the next state.
- Simplifies protocol implementation.

Key Point

Both connectionless and connection-oriented transport protocols can be represented using FSMs.

Components of an FSM

An FSM consists of four basic components.

① States

- Represents the current condition of the protocol.

② Events

- External occurrences such as packet arrival or timeout.

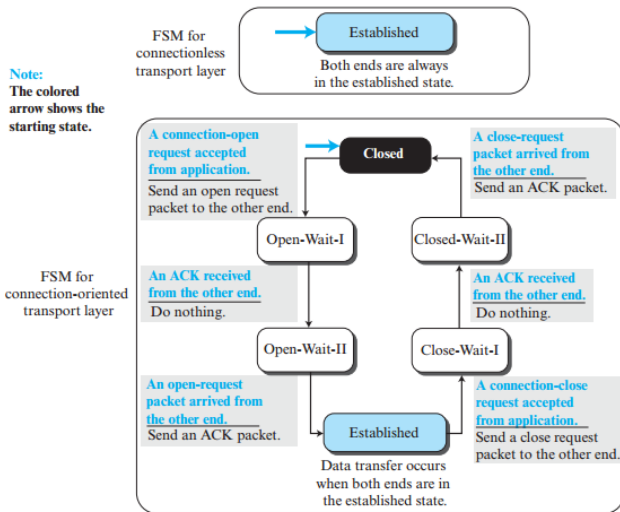
③ Actions

- Operations performed when an event occurs.

④ State Transition

- Movement from one state to another.

Figure 3.16 Connectionless and connection-oriented service represented as FSMs



Connectionless Transport Layer

- Only one state:

Established

- Always ready to send and receive packets.
- No connection setup.
- No connection termination.

Connection-Oriented Transport Layer

- Multiple states.
- Requires connection establishment.
- Data transfer occurs only after connection establishment.
- Connection is terminated after communication.

Connection-Oriented FSM States

Connection Establishment

- 1 Closed
- 2 Open-Wait-I
- 3 Open-Wait-II
- 4 Established

Connection Termination

- 1 Close-Wait-I
- 2 Close-Wait-II
- 3 Closed

Data Transfer

Packets are exchanged only when both sender and receiver are in the **Established** state.

Chapter Summary

Flow Control

- Producer–Consumer
- Push vs Pull
- Buffers
- Prevents overflow

Error Control

- Sequence Numbers
- ACKs
- Duplicate Detection
- Retransmission

Transport Services

- Connectionless
- Connection-Oriented
- Reliability
- Ordered Delivery

FSM

- States
- Events
- Actions
- Transitions

Key Takeaways

- Flow Control balances sender and receiver speeds.
- Error Control provides reliable packet delivery.